

DCMC: Study Project WS 25/26

In the course "Compiler Construction" you have the chance to work on a study project closely related to the topics of the lectures and exercises

- Assessment mode:
 - Source code archive with the study project implementation
 - 2-5 pages description of your project (structure, characteristics, examples, etc.)
 - Short presentation (only live demo and Q&A, 10-15 mins, no slides needed) of your work
- Evaluation: Performance verification (graded)
- Deadline: Sun, 2026-02-01 23:59:59
- Submission: Send the archive with source code and description via E-Mail to florian.heinz@oth-regensburg.de

Task:

Your project should demonstrate your understanding of the lecture topics.

Mandatory parts:

- Building a lexicographical scanner
- Building a syntactical parser
- Creating an abstract syntax tree
- Then choose:
 - Create an interpreter to execute the program that is represented by the Abstract Syntax Tree by traversing it
 - Compile the Abstract Syntax Tree to machinecode (e.g. bytecode for a vm)

Optional parts:

- Post-process your abstract syntax tree for optimizations
- Run further semantic checks on the abstract syntax tree (e.g. type checks or similar)

General task description:

Your task is to create a programming language. There are no high requirements regarding the programming language design, the focus is originality, lexing, parsing, abstract syntax trees and interpreting/compiling.

The programming language should be general purpose, three programming tasks should be fulfilled with it for demonstration. You may of course optimize it for specific needs.

The syntax and semantics of the programming language should be very individual and should deliberately deviate from other programming languages, specifically your syntax should not

be similar to the syntax of the language C! This includes, but is not restricted to:

- Number formats
- String literals
- Identifiers
- Operators
- Keywords
- Control structures
- Functions
- Variable handling
- Data types
- ... and so on

The above listed features have to be implemented in one or another way.

Some ideas for possible optional features include:

- Static type system (checked at compile time)
- Advanced control structures (for example, a while-loop that has some kind of else-branch, if the condition is not fulfilled already before the first iteration)
- Special syntax for iterating over a range with specific step widths
- Variadic functions
- More individual syntactic elements for special purposes

Compiling:

If you choose the way to compile your program to bytecode, you can use the virtual machine VM3 introduced in the lecture

You can get it by cloning with GIT:

```
$ git clone https://github.com/fwheinz/vm3
$ cd vm3
$ make
```

The project contains an example lexer and parser/bytecode compiler **prolang**. You can use it as reference to get an idea, how certain constructs can be implemented, but it is in general unsuitable as a base for your own project, so do not use it except for reference purposes! Start the implementation from scratch or based on the past exercises, if you have accomplished these. You can change the name of the target in the Makefile to reflect your own program name.

Feel free to extend and modify the virtual machine. Implement custom native functions and/or opcodes in **custom.c**, so you can still update the project without conflicts. But you can also modify other parts if you feel it is necessary.

Development:

Please use the *OTH GitLab* for development (<https://gitlab.oth-regensburg.de/>). This is necessary to record your development progress and show, that your project was created from a continuous development process and not from copied or generated code.

Important: Do regular commits and push them to your repository (git commit, git push), ideally once per coding step but *at least* once per day when you made changes to your project. Use meaningful and accurate git commit messages.

Invite me to your project, so I can help you quickly if you run in any serious problems. Read-only access is sufficient.

Demonstration:

Implement four programs with your programming language:

- "Game of Life"
- "Run Length Decoder"
- "Black Jack"
- "Language Demo"

Game of Life: Implement Conway's Game of Life

```
+-----+
|      |      |
|  #    #    |
|  #    #    |
|  #   ###   |
|                ###
|      |      |
+-----+
```

The "Game of Life" is a simple cellular automaton. Each

- living cell stays alive, if 2 or 3 neighboring cells are alive
- living cell dies if it has less than 2 (isolation) or more than 3 (overpopulation) live neighbors
- dead cell becomes alive if it has exactly 3 live neighbors

More information is available at https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life
Minimal requirements are accurate calculation and visualization of the field for an arbitrary number of iterations. Details are up to you, i.e.:

- How is the initial field specified (e.g. hardcoded, definition file, user input, etc)
 - How can the user interact with the field during the iterations
 - How exactly is the field visualized
- and much more. Extras are appreciated.

Run Length Decoder: The user enters a string containing the letters a-z and additionally a run length specifier like `n[...]` that repeats everything between the square brackets *n*-times. These specifiers can be nested in arbitrary depth. You can assume, that *n* is only a single digit number. Examples:

- `5[ab] → ababababab`
- `3[hello2[world]] → helloworldworldhelloworldworldhelloworldworld`
- `2[a3[b2[cd]]] → abcdcdcbcdcdcbcdcdabcdcdcbcdcdcbcdcd`

This problem should be solved recursively!

Black Jack: Play black jack against the computer. The user receives two random cards (2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K, A), color of cards does not matter. Each card has the value printed on it. J, Q, and K have value 10. A has value 11, except if the total sum would be above 21, then it has value 1. The user tries to get as close as possible to 21 by requesting additional cards. The user can stop drawing cards ("stay") anytime; if the user exceeds 21 ("busts"), he loses. After that, the computer opponent tries to get a higher value than the user, the same rules apply. Keep

track of wins and losses.

Language demo: Your own program that demonstrates the specifics of your language

Bonus: Additional non-trivial programs give bonus points

Final words:

- If you have your own cool idea for this study project, that considerably deviates from the task described here, write me an e-mail and I will consider it!
- Do your task step by step, starting with the lexer, continuing with the parser, the abstract syntax tree and finally executing the program one way or another. This is not a trivial task, so don't get frustrated easily when you are stuck somehow.
- If you use VM3 and you can rule out own errors, chances are you might have hit a bug. Please report it to me and I will try to fix it
- There is help:
 - Have a look at the example implementations provided in the lecture slides, ELO and the github repository of VM3
 - Ask for support via email. You can either attach your code or send a git repository link. Provide all information necessary to reproduce the problem you have.
- Please do not use AI-generated code or code from other persons in any way. Be prepared for detailed questions on your code at the presentation session; if there are serious doubts that you wholly wrote it yourself, you cannot pass the exam successfully.

Good Luck!